Neuroevolution of Compact Search Heuristics in Sokoban

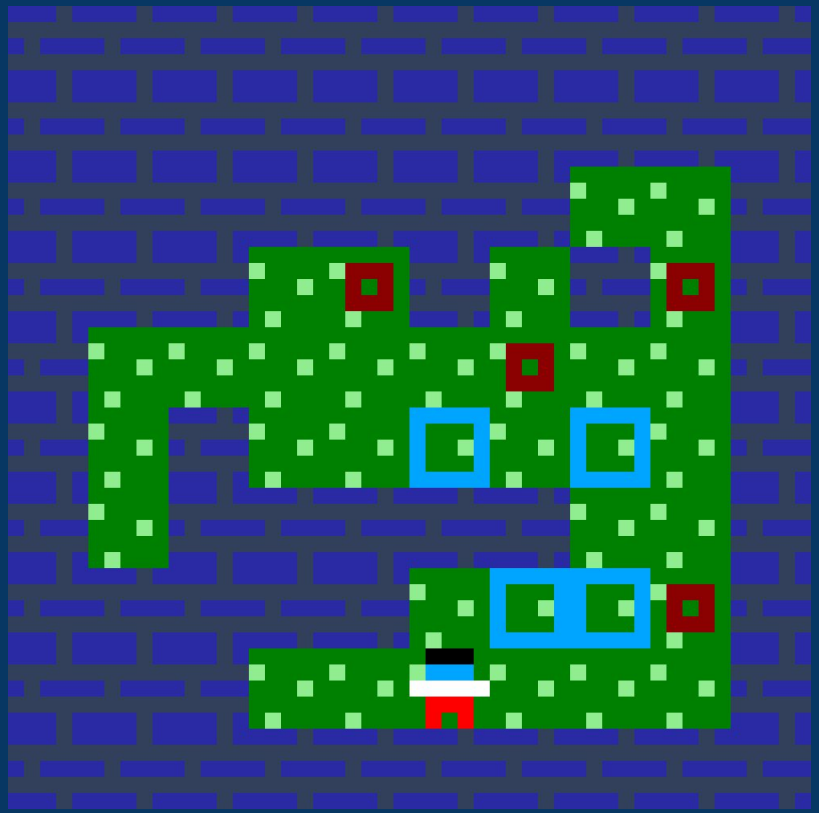
Matthew Lee
Professor Gary B. Parker, Jim O'Connor
Honors Thesis Semester 2

Roadmap

1. Sokoban is an interesting problem to solve
2. Background knowledge: A*, CNNs, NEAT, CoDeepNEAT
3. Prior solvers: from rule-based to learned heuristics
4. Methodology
5. Results
6. Conclusion and Future Work

What is Sokoban?

- Popular 2d grid puzzle
- Goal: Push boxes onto goals
- Sokoban is an interesting problem in the domain of computational intelligence because of its NP-Hard, P-Space-Completeness



Difficulty of Solving Sokoban Puzzle

- Irreversible: Player can only push boxes
- Sparse rewards: Pushing boxes does not grant any reward signal
- Long planning horizons: Boxes need to be pushed for long time with no apparent reward for solution
- Combinatorial state space: Even with branching factor of 4, because of the depths of the solution, the state spaces explode

Can neuroevolution discover compact learned heuristic functions?

Background

- Heuristic function
  - How long does it take from current state to the goal? (Cost-to-Go)
- A* Search
  - Best-first search using heuristic function
  - Given an admissible heuristic, will find solution of any deterministic planning problems
  - Admissible means that the heuristic function never overestimates the value of a state
- Neuroevolution
  - Evolutionary method of training neural networks. Can be about training weights or training architectures
- Gradient descent
  - Optimization of weights using a derivative of the loss function. Based on given loss, we can mathematically calculate the delta weights

Can neuroevolution discover compact learned heuristic functions?

With the background, we can rephrase the question as follows:

Can we evolve neural networks for use in A* search that will find solutions for unseen Sokoban puzzles, in a way that is weight efficient?

NEAT

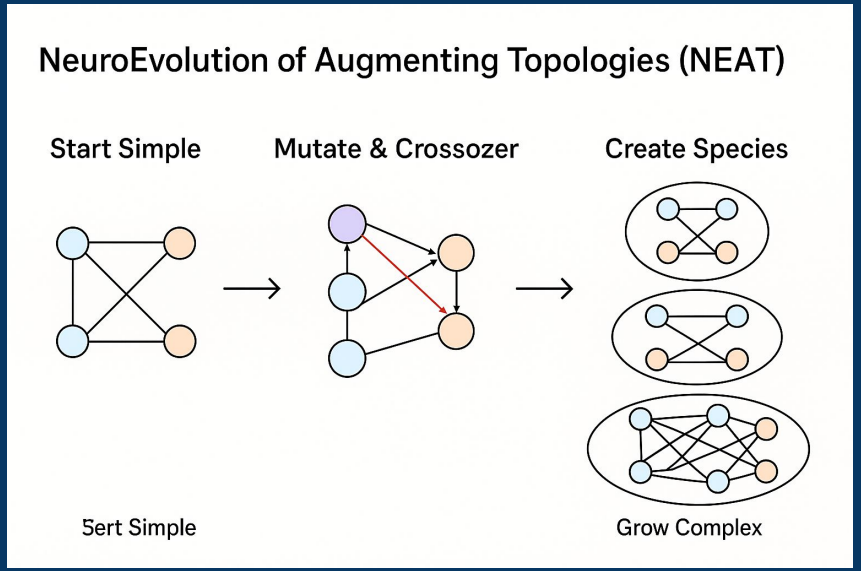
NeuroEvolution of Augmenting Topologies (by Stanley and Mikkulainen)

Evolves both weights and topology

Starts minimal, grows complexity through mutation

- Adds nodes
- Adds connections
- Perturb weights

Speciation protects new innovation in genomes



CoDeepNEAT

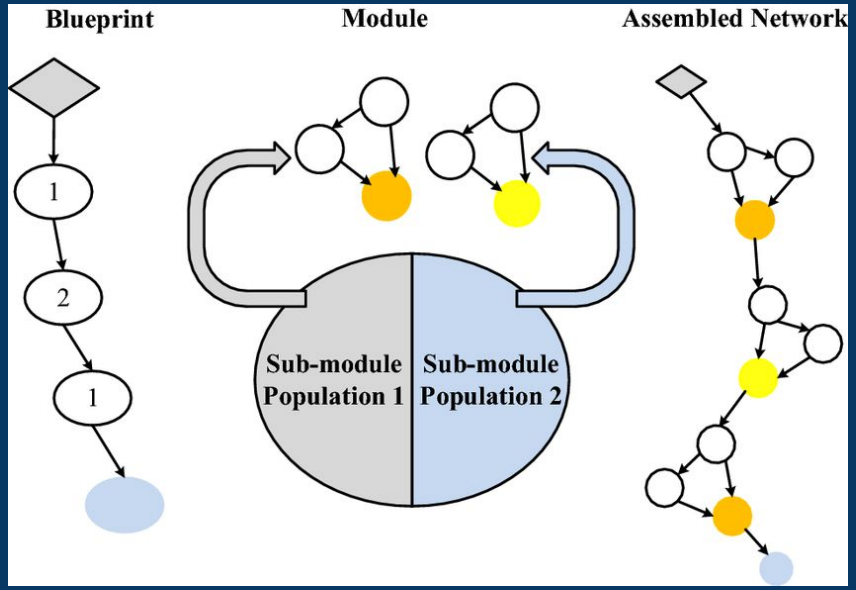
Evolution of architecture, trained weights

Coevolution:

- Modules
- Blueprints

Assemble -> train by gradient descent -> score

Hybrid method that incorporates gradient descent into the evolutionary framework



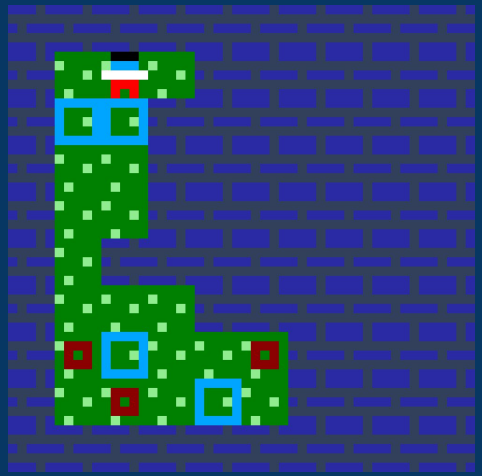
Baseline heuristic for A*

$$f(n) = g(n) + h(n)$$

We compare NEAT and CoDeepNEAT heuristic functions against the following heuristic functions:

Manhattan Distance: Sum of all distances between each boxes to each goals

CNN (based on DeepcubeA): DCNN trained with reverse trajectories from Breadth First Search solution

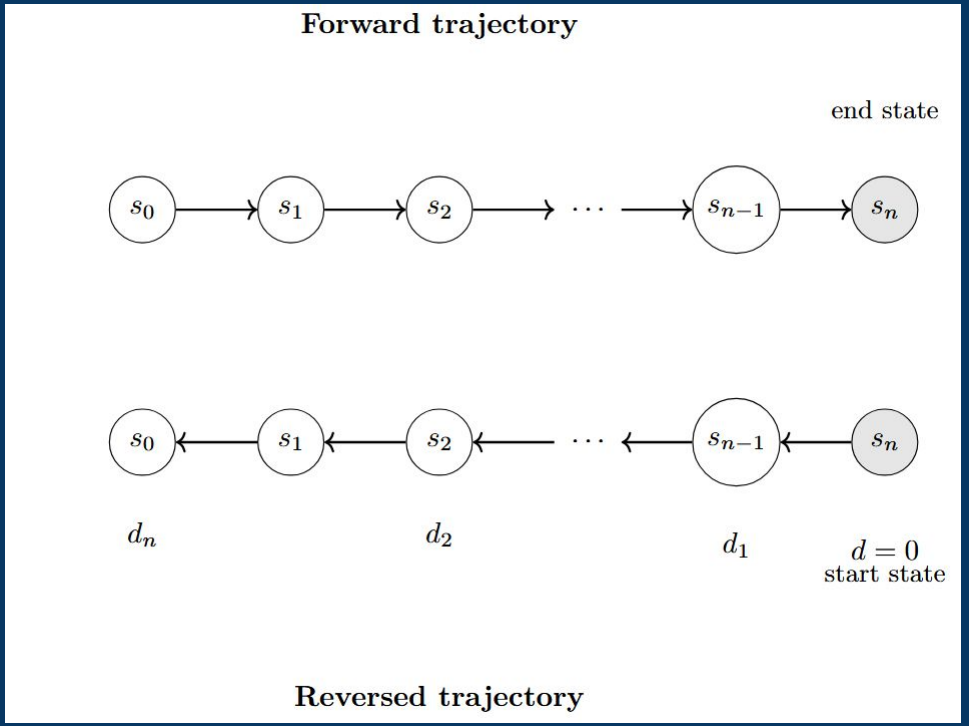


Training data for the heuristic functions

Reverse Trajectory:

Breadth-First Search generates a solution: Forward Trajectory

By reversing the trajectory we know the distance of each state from the goal state: cost-to-go = heuristic function



Results

Heuristic	Parameters	Solve Rate
Manhattan distance (minimum baseline)	0	87.4% (257/294)
NEAT (average across 9-config sweep)	∼25	83.9% (246.8/294)
CoDeepNEAT	263,361	88.8% (261/294)
DeepCube CNN cost-to-go (upper-limit baseline)	2,031,809	100.0% (294/294)

Table 6.1: Solve rate of each heuristic on the 294 held-out medium-split Boxoban levels under A* search with a 100,000 node expansion budget. The NEAT row reports the mean solve rate across the nine-configuration hyperparameter sweep (Appendix A.4); per-configuration rates ranged from 82.7% to 84.7%. Manhattan distance is the minimum baseline and the DeepCube-style CNN cost-to-go network is the upper-limit baseline.

Future Work

CNN proved to be strong but requires many parameters

-> CoDeepNEAT can incorporate CNN for reduced architecture with optimized parameter count

Thank you